

第1章-5: Jetpack Compose 列表与状态管理

(小白专属)

讲师寄语：各位小伙伴，之前咱们学了Column布局能垂直排列元素，但如果要做微信聊天列表、新闻列表这种“海量数据”的界面，直接用Column会卡顿到崩溃！今天咱们就学两个核心技能：一是用LazyColumn高效实现列表，二是用remember+mutableStateOf搞定“数据变UI更”的状态管理，这俩是Compose开发的“黄金搭档”，跟着敲代码绝对能学会！

一、列表救星：LazyColumn 懒加载列表

先思考一个问题：如果用普通Column展示1000条数据，会发生什么？——Column会一次性把1000条数据全加载到内存，不管屏幕上能不能看到，手机CPU和内存直接“过载”，界面就会卡顿、掉帧。而LazyColumn的核心是“懒加载”，只加载屏幕上“看得见”的内容，看不见的全不加载，完美解决卡顿问题。

1. LazyColumn 核心原理：按需加载

 比喻理解：LazyColumn就像“书店的书架”，你站在原地只能看到眼前的10本书（屏幕内容），书架后面的书（屏幕外内容）不会提前搬到你眼前；只有当你往前走（滑动屏幕），后面的书才会被搬过来，前面的书会被收起来。这样既节省空间（内存），又节省力气（CPU）。

2. 入门示例：写一个基础的LazyColumn列表

咱们先做一个“100条数据的简单列表”，代码直接复制到AS就能运行：

Code block

```
1  import androidx.compose.foundation.lazy.LazyColumn
2  import androidx.compose.foundation.lazy.items
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5  import androidx.compose.ui.Modifier
6  import androidx.compose.ui.unit.dp
7  import androidx.compose.foundation.layout.padding
8
9  @Composable
10 fun BasicLazyColumn() {
11     // 1. 模拟100条列表数据（实际开发中可能来自网络或数据库）
```

```

12     val dataList = List(100) { index ->
13         "这是第${index + 1}条列表数据"
14     }
15
16     // 2. 懒加载列表核心代码
17     LazyColumn(
18         modifier = Modifier.padding(16.dp),
19         // 可选：设置列表项之间的垂直间距
20         verticalArrangement =
21             androidx.compose.foundation.layout.Arrangement.spacedBy(8.dp)
22     ) {
23         // 3. 遍历数据生成列表项（items是LazyColumn的专属遍历方法）
24         items(dataList) { itemText ->
25             // 4. 每个列表项的UI（这里用Text，实际可换成卡片、图片等）
26             Text(
27                 text = itemText,
28                 modifier = Modifier.padding(8.dp),
29                 fontSize = 16.sp
30             )
31         }
32     }

```

3. 关键代码解析（小白必看）

- **List(100) {}**：Kotlin的便捷方法，生成包含100个元素的列表，index是从0开始的索引，所以用index+1得到“第N条”。
- **LazyColumn {}**：和普通Column不同，LazyColumn的花括号里不是直接写子元素，而是写“列表项的生成规则”。
- **items(dataList)**：LazyColumn提供的“数据遍历工具”，作用是把dataList里的每个元素，都转换成一个列表项。它会自动管理列表项的加载和回收，不用咱们手动处理。
- **itemText**：dataList里的单个元素（比如“这是第1条列表数据”），咱们直接用它构建列表项UI即可。

4. 进阶：带点击事件的列表项

实际开发中列表项都能点击（比如点击进入详情页），给列表项加点击事件超简单，用Modifier.clickable就行：

Code block

```

1  @Composable
2  fun ClickableLazyColumn() {
3      val dataList = List(100) { "第${it+1}条可点击数据" }

```

```
4
5     LazyColumn(modifier = Modifier.padding(16.dp)) {
6         items(dataList) { itemText ->
7             Text(
8                 text = itemText,
9                 modifier = Modifier
10                    .padding(8.dp)
11                    .fillMaxWidth() // 让文字占满宽度，点击范围更大
12                    .clickable {
13                        // 点击事件：打印当前点击的内容
14                        println("你点击了: $itemText")
15                    },
16                 fontSize = 16.sp
17            )
18        }
19    }
20 }
```

运行后点击任意列表项，AS的Logcat里就能看到打印的内容，是不是很简单？

5. 避坑点：LazyColumn vs Column

很多小白会混淆这两个布局，记住以下两点就能分清：

布局类型	适用场景	核心特点
Column	少量元素（比如3-5个按钮、文字）	一次性加载所有元素，简单但不适合大量数据
LazyColumn	大量元素（列表、数据展示）	按需加载，节省内存，避免卡顿

二、状态管理核心：remember + mutableStateOf

学完列表，咱们遇到一个新问题：如果想做“点击按钮让列表项变色”“输入文字实时显示”这种“数据变UI跟着变”的功能，该怎么实现？这就需要“状态管理”，而remember和mutableStateOf是Compose中最基础、最常用的状态管理组合。

1. 先搞懂：什么是“状态”？

状态（State）就是APP中“会变化的数据”，比如：

- 按钮的“是否被点击”（布尔值：true/false）
- 输入框的“文字内容”（字符串：“你好”“Compose”）
- 开关的“是否打开”（布尔值：true/false）

Compose的核心思想是“状态驱动UI”——你只需要管理状态，UI会自动根据状态变化而更新，不用像XML那样手动写“textView.setText()”这种代码。

2. 为什么需要 remember 和 mutableStateOf?

咱们先看一个“反例”，理解没有它们会发生什么：

Code block

```
1  @Composable
2  fun BadStateDemo() {
3      // 定义一个“点击次数”变量
4      var clickCount = 0
5
6      Column(modifier = Modifier.padding(16.dp)) {
7          Text(text = "点击次数: $clickCount")
8          Button(onClick = {
9              // 点击按钮，让次数+1
10             clickCount++
11             println("点击次数: $clickCount") // 日志会增加，但UI不变
12         }) {
13             Text(text = "点我加1")
14         }
15     }
16 }
```

运行后你会发现：点击按钮时，Logcat里的clickCount会增加，但屏幕上的文字始终显示“点击次数：0”。这是因为Compose的函数会“重组”（更新UI时重新执行函数），每次重组都会把clickCount重新赋值为0，状态没被保存住。

而remember和mutableStateOf就是来解决这个问题的：

- **mutableStateOf**：把普通数据包装成“可观察的状态”，当数据变化时，会通知Compose“该更新UI了”。
- **remember**：在Compose重组时“记住”状态的值，不会让状态被重新初始化（避免上面的“重置”问题）。

3. 入门示例：实现“点击计数”功能

用remember+mutableStateOf修改上面的反例，实现“点击按钮次数实时更新”：

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.material3.Button
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
```

```

5  import androidx.compose.runtime.mutableStateOf
6  import androidx.compose.runtime.remember
7  import androidx.compose.ui.Modifier
8  import androidx.compose.ui.unit.dp
9  import androidx.compose.foundation.layout.padding
10
11  @Composable
12  fun GoodStateDemo() {
13      // 核心：用remember记住mutableStateOf包装的状态
14      val clickCount = remember { mutableStateOf(0) }
15
16      Column(modifier = Modifier.padding(16.dp)) {
17          // 读取状态：用.value获取包装的数据
18          Text(text = "点击次数: ${clickCount.value}", fontSize = 18.sp)
19
20          Button(onClick = {
21              // 修改状态：给.value赋值
22              clickCount.value++
23          }) {
24              Text(text = "点我加1")
25          }
26      }
27  }

```

运行后点击按钮，屏幕上的“点击次数”会实时增加，这就是状态管理的魔力！

4. 关键代码解析（小白必背）

- **val clickCount = remember { mutableStateOf(0) }:**
 - 外层remember：保证重组时clickCount不会被重新创建，状态值被“记住”。
 - 内层mutableStateOf(0)：把初始值0包装成“可观察状态”，0是初始值。
- **clickCount.value**：mutableStateOf包装的数据，必须通过.value来读取和修改，这是固定写法！
- **clickCount.value++**：修改状态值，此时mutableStateOf会通知Compose“状态变了”，Compose会重新执行GoodStateDemo函数，更新UI。

5. 简化写法：by 关键字

每次都写clickCount.value有点麻烦，Compose支持用by关键字简化写法，需要先导入一个包：

```

1  // 必须导入这个包才能用by
2  import androidx.compose.runtime.getValue
3  import androidx.compose.runtime.setValue
4
5  @Composable
6  fun SimplifiedStateDemo() {
7      // 简化: 用by代替val, 不用再写.value
8      var clickCount by remember { mutableStateOf(0) }
9
10     Column(modifier = Modifier.padding(16.dp)) {
11         // 直接用clickCount, 不用.value
12         Text(text = "点击次数: $clickCount", fontSize = 18.sp)
13         Button(onClick = {
14             // 直接修改clickCount, 不用.value
15             clickCount++
16         }) {
17             Text(text = "点我加1")
18         }
19     }
20 }

```

效果和之前完全一样，但代码更简洁了。小白推荐用这种写法，更符合咱们平时写变量的习惯。

三、黄金搭档：LazyColumn + 状态管理

实际开发中，列表和状态管理总是一起用的，比如“点击列表项变色”“删除列表项”。咱们做一个“可删除的列表”，巩固今天的所有知识点。

实战：实现“长按删除列表项”功能

Code block

```

1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.clickable
3  import androidx.compose.foundation.layout.Column
4  import androidx.compose.foundation.layout.fillMaxWidth
5  import androidx.compose.foundation.layout.padding
6  import androidx.compose.foundation.lazy.LazyColumn
7  import androidx.compose.foundation.lazy.items
8  import androidx.compose.foundation.shape.RoundedCornerShape
9  import androidx.compose.material3.Text
10 import androidx.compose.runtime.Composable
11 import androidx.compose.runtime.getValue
12 import androidx.compose.runtime.mutableStateOf
13 import androidx.compose.runtime.remember
14 import androidx.compose.runtime.setValue

```

```

15 import androidx.compose.ui.Modifier
16 import androidx.compose.ui.graphics.Color
17 import androidx.compose.ui.unit.dp
18 import androidx.compose.ui.unit.sp
19
20 @Composable
21 fun DeletableLazyColumn() {
22     // 1. 用状态管理列表数据 (因为列表会被删除, 数据会变化)
23     var dataList by remember {
24         mutableStateOf(
25             List(20) { "可删除列表项 ${it+1}" }
26         )
27     }
28
29     Column(modifier = Modifier.padding(16.dp)) {
30         Text(text = "长按列表项删除", fontSize = 18.sp, modifier =
Modifier.padding(bottom = 8.dp))
31
32         // 2. 懒加载列表
33         LazyColumn(verticalArrangement =
androidx.compose.foundation.layout.Arrangement.spacedBy(8.dp)) {
34             // 3. 遍历状态列表
35             items(dataList) { itemText ->
36                 // 4. 列表项: 带背景的卡片, 长按删除
37                 Text(
38                     text = itemText,
39                     modifier = Modifier
40                         .fillMaxWidth()
41                         .background(Color.LightGray, shape =
RoundedCornerShape(8.dp))
42                     .padding(16.dp)
43                     // 长按事件: 删除当前列表项
44                     .clickable(
45                         onClick = {}, // 空的点击事件 (必须写, 否则长按不生效)
46                         onLongClick = {
47                             // 修改状态: 过滤掉当前点击的itemText
48                             dataList = dataList.filter { it != itemText }
49                         }
50                     ),
51                     fontSize = 16.sp
52                 )
53             }
54         }
55     }
56 }

```

实战代码解析

- **列表数据用状态管理**：dataList是用remember+mutableStateOf包装的，因为删除操作会改变列表数据，需要通知UI更新。
- **长按事件删除逻辑**：用filter方法过滤掉当前点击的itemText，生成新的列表并赋值给dataList，状态变化后LazyColumn会自动更新UI，删除对应的列表项。
- **clickable的小技巧**：如果只需要长按事件，onClick参数必须写一个空实现，否则长按事件不会生效，这是Compose的一个小约定。

四、本章小结（小白必背）

1. **LazyColumn**是实现大量数据列表的核心，用items遍历数据，按需加载避免卡顿，适合做聊天列表、新闻列表等。
2. **状态**是APP中会变化的数据，Compose用“状态驱动UI”，状态变了UI自动更。
3. **remember+mutableStateOf**是基础状态管理组合：mutableStateOf包装可观察状态，remember在重组时记住状态。
4. 用by关键字可以简化状态的读写（不用写.value），但要导入getValue和setValue包。
5. 列表和状态管理结合使用，能实现“删除、修改列表项”等交互功能。

 课后作业：基于今天的“可删除列表”，修改代码实现“点击列表项变色”功能——定义一个状态记录“当前选中的列表项文本”，选中的项背景色变成蓝色，未选中的是灰色。动手实操才能真正掌握！

（注：文档部分内容可能由 AI 生成）